Project Deliverable

University of Sialkot

Department of Software Engineering

Faculty of Software Engineering

Web Engineering

Guiding Principles: Smart Diet Planner (TandooriFit)

Submitted To:

Sir Museb Khalid

Dated: 15/07/2026

By:

Muhammad Jasim 23010101-054

Atiqa Asif 23010101-016

Introduction

Smart Diet Planner is a client-side web application designed to help individuals plan and manage a healthy diet in a simple and personalized way. It allows users to calculate essential health metrics such as BMI, BMR, TDEE, and daily calorie requirements based on their age, gender, body measurements, activity level, and fitness goals. Based on these results, the application assists users in creating a structured daily meal plan with macro-nutrient targets and flexible meal distribution, making diet planning more practical and user-friendly.

The application is useful for users who want a clear and guided approach to nutrition without complex manual calculations. Many people find it difficult to estimate calories and macros or to design balanced meals that align with their fitness goals. Smart Diet Planner addresses this problem by providing automated calculations, customizable meal planning, and a flexible food database that can be adapted or extended in future versions. By focusing on simplicity, clarity, and modular design, the application lays a strong foundation for future enhancements such as online data storage, user accounts, and dynamic food databases.

Project Features

1. **BMI Calculator**
Calculates Body Mass Index using user-provided height and weight and categorizes the result to help users understand their current health status.

2. **Daily Calorie Calculator**
Computes daily calorie requirements based on BMR, activity level, and fitness goals such as weight maintenance, loss, or gain.

3. **Meal Planner**
Generates a structured daily meal plan according to calorie targets, selected diet style, and number of meals per day.

4. **Macro-Nutrient Breakdown**
Provides detailed information about daily protein, carbohydrate, and fat requirements to support balanced nutrition.

5. **Flexible Food Database**
Allows users to manage food items and nutritional values, with support for modifying or extending the food list as needed.

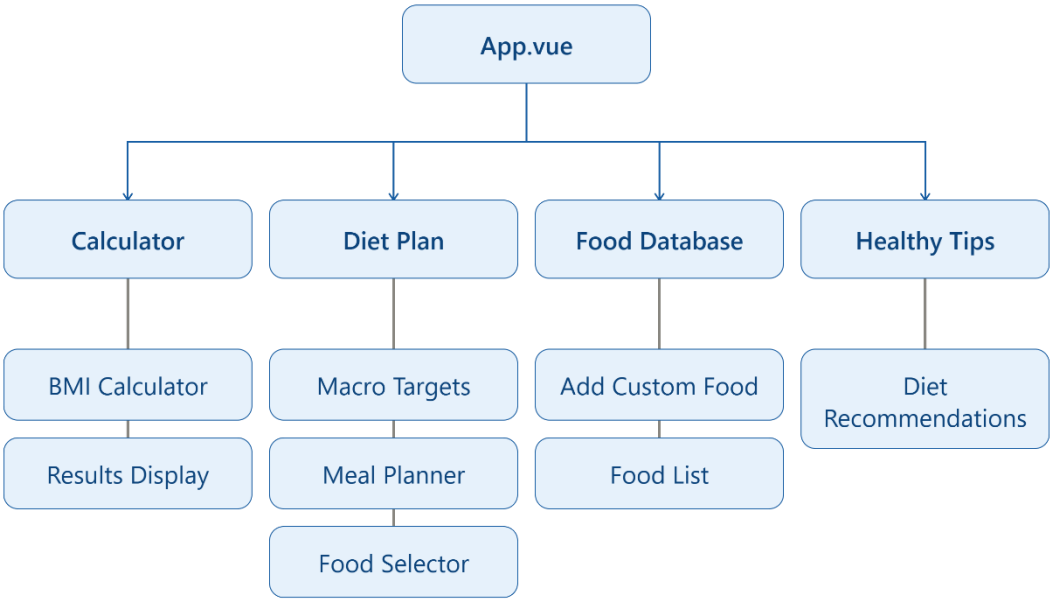
6. **Diet Recommendations**
Offers healthy dietary tips and guidance to promote better eating habits and informed food choices.

7. **User-Friendly Dashboard**
Presents calculated results and meal plans in a clear and organized interface for easy monitoring.

8. **Responsive Single Page Application**

Ensures smooth navigation and usability across different screen sizes using client-side routing.

Architecture Tree:



Vue Router

1. / → **CalculatorView**
   (BMI, BMR, TDEE, and daily calorie calculator)
2. /plan → **PlanView**
   (Daily meal planner with macro breakdown and meal management)
3. /database → **DatabaseView**
   (Food database management with add/edit and macro estimation)
4. /tips → **TipsView**
   (Healthy diet tips and nutrition guidance)

Components

1. **App.vue** → Root component that manages layout, navigation, and shared application state
2. **BmiCalculator.vue** → Takes user input for BMI, BMR, TDEE, and calorie calculations
3. **BmiResults.vue** → Displays calculated health results such as BMI category and daily calorie needs
4. **CalculatorView.vue** → Combines calculator input and results into one page
5. **PlanView.vue** → Handles meal planning, meal updates, and macro tracking
6. **DatabaseView.vue** → Manages food database and custom food entries

7. **TipsView.vue** → Displays healthy diet tips and nutrition guidance
8. **macroEstimator.js (utility)** → Automatically estimates food macros based on name keywords
9. **router/index.js** → Manages navigation between different views in the SPA

Props and Emits

In Vue.js, **props** and **emits** are used for communication between components.

Props are used to send data from a parent component to a child component. This allows child components to display or use data provided by the parent.

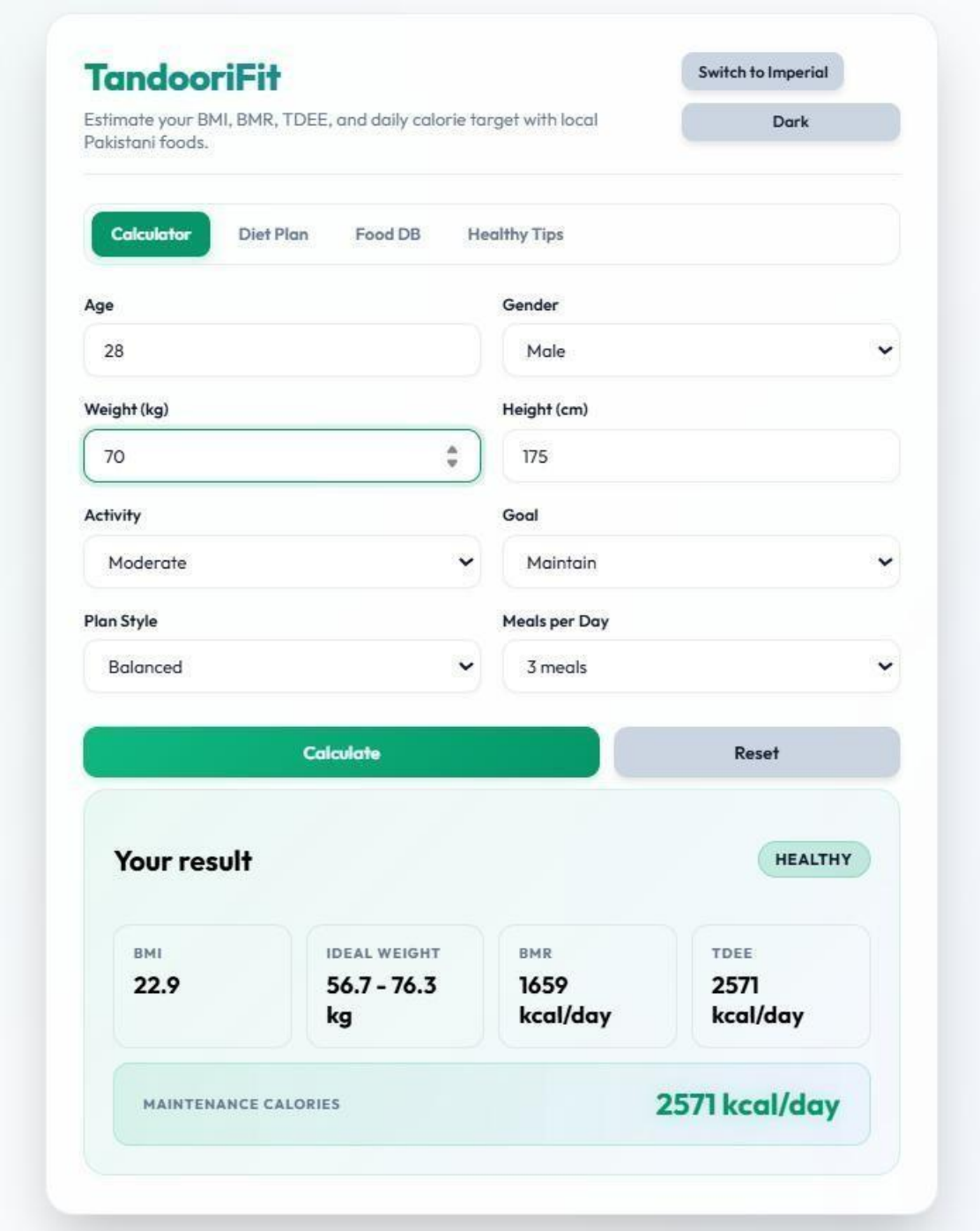
Emits are used when a child component wants to send information or trigger an action in the parent component. It works like sending a message upward.

Example code of BmiCalculator Component including props and emitters:

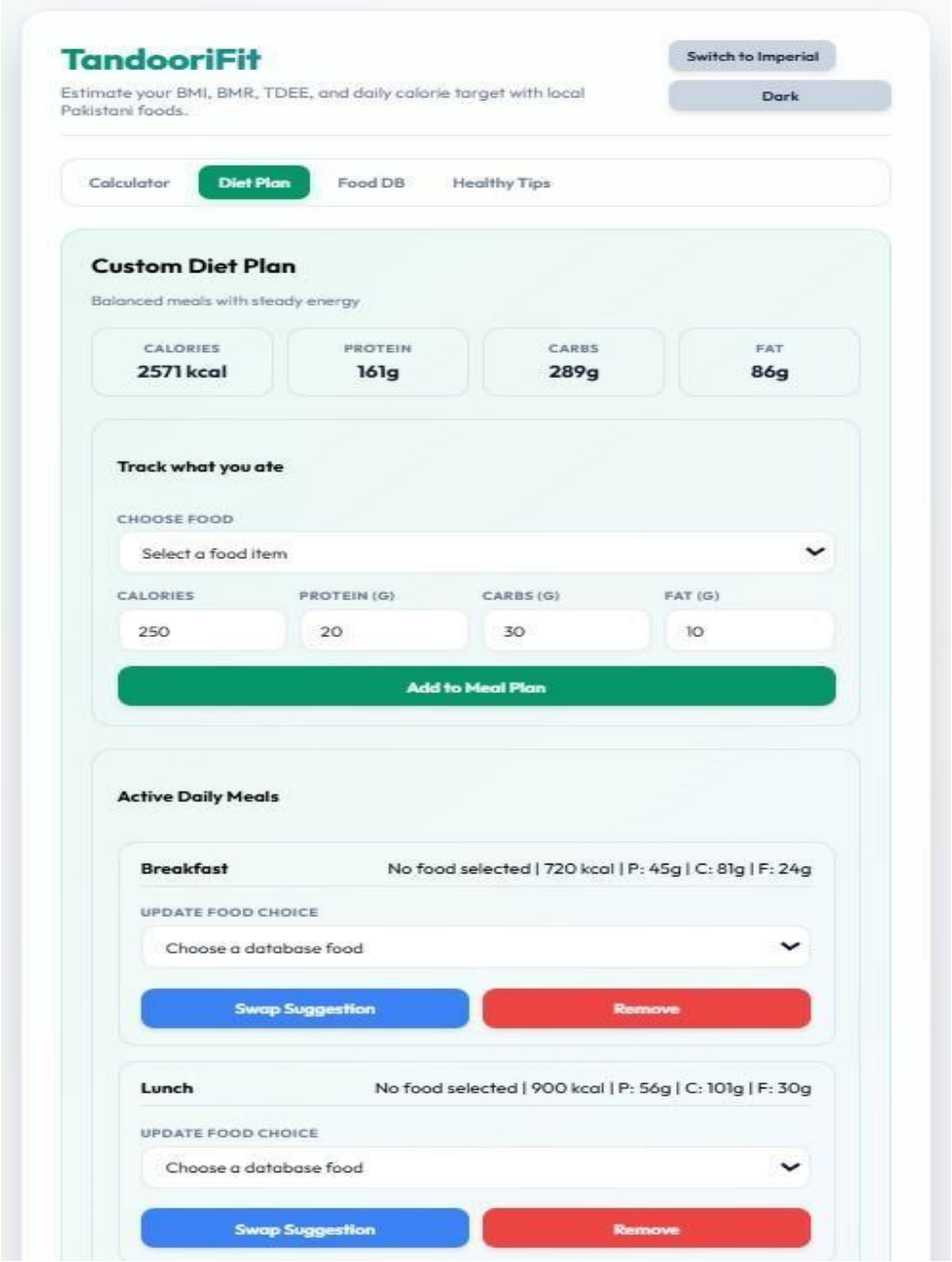
```
const props = defineProps({
  units: String,
  age:          Number,
gender:          String,
weightKg:     Number,
weightLbs:    Number,
heightCm:     Number,
feet: Number,   inches:
Number,
  activity: String,
goal: String,
planStyle: String,
  mealsPerDay: Number
});
const emit = defineEmits([
  'update:units', 'update:age', 'update:gender', 'update:weightKg',
  'update:weightLbs', 'update:heightCm', 'update:feet', 'update:inches',
  'update:activity', 'update:goal', 'update:planStyle', 'update:mealsPerDay',
  'calculate', 'reset'
]);
```

Screenshots:

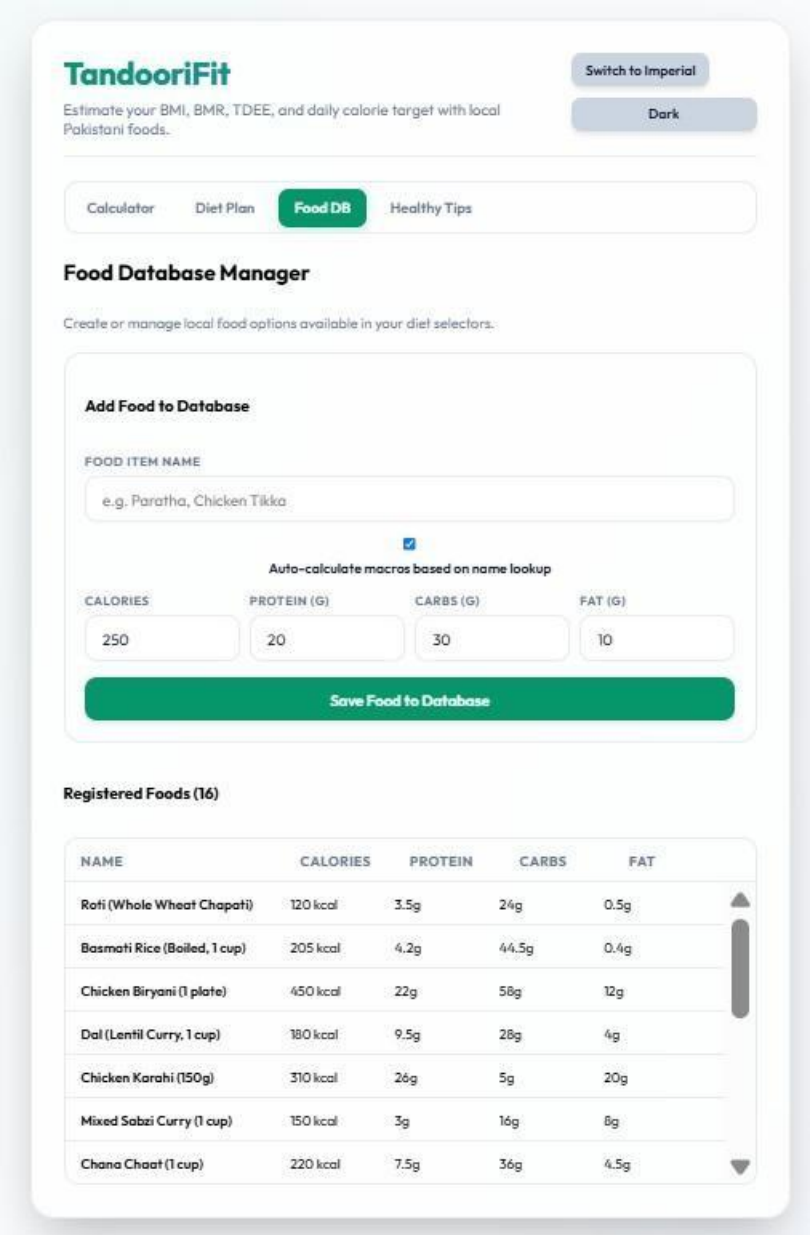
Calculator:



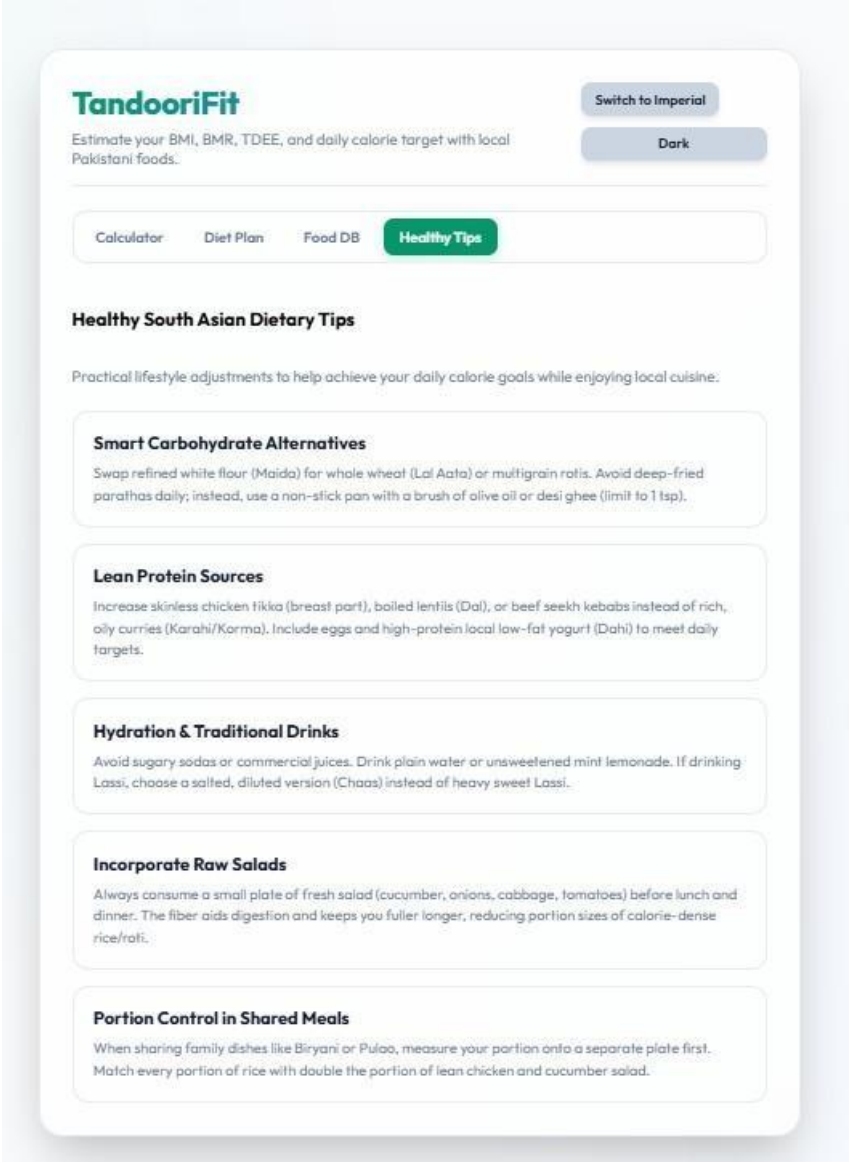
Diet Plan:


TandooriFit
Estimate your BMI, BMR, TDEE, and daily calorie target with local Pakistani foods.
Switch to Imperial
Dark
Calculator
Diet Plan
Food DB
Healthy Tips
Custom Diet Plan
Balanced meals with steady energy
CALORIES
2571 kcal
PROTEIN
161g
CARBS
289g
FAT
86g
Track what you ate
CHOOSE FOOD
Select a food item
CALORIES
PROTEIN (G)
CARBS (G)
FAT (G)
250
20
30
10
Add to Meal Plan
Active Daily Meals
Breakfast
No food selected | 720 kcal | P: 45g | C: 81g | F: 24g
UPDATE FOOD CHOICE
Choose a database food
Swap Suggestion
Remove
Lunch
No food selected | 900 kcal | P: 56g | C: 101g | F: 30g
UPDATE FOOD CHOICE
Choose a database food
Swap Suggestion
Remove

Food Database:



Healthy Tips:



Backend

The backend of this project is the server-side part of the Smart Diet Planner application. It is built using Node.js and Express, and it handles communication between the frontend and the MongoDB database. Its main responsibilities are to start the server, manage API requests, connect to the database, and store or retrieve the diet planner data such as user details, meal plan, results, and food items. In simple terms, the backend acts as the bridge that allows the app to save and load data reliably.

1. **backend/index.js** →Main server file that starts the backend.
2. **backend/config/db.js** → Connects the app to MongoDB.
3. **backend/routes/health.js** → Health check API route.
4. **backend/routes/state.js** → API route for saving and loading app state.
5. **backend/models/DietPlannerState.js** → Schema for storing the overall planner state.
6. **backend/models/FoodItem.js** → Schema for storing food item data.

7. **backend/models/FoodEntry.js** → Schema for storing food entries associated with a state.
8. **backend/models/MealPlanItem.js** → Schema for storing meal plan entries.
9. **backend/models/Result.js** → Schema for storing BMI/BMR/TDEE calculation results.

Example code of backend (FoodEntry.js)

```
import mongoose from 'mongoose';

const FoodEntrySchema = new mongoose.Schema(
  {
    stateId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'DietPlannerState',
      required: true,
    },
    name: { type: String, required: true, trim: true },
    calories: { type: Number, default: 0 },
    protein: { type: Number, default: 0 },
    carbs: { type: Number, default: 0 },
    fat: { type: Number, default: 0 },
  },
  { timestamps: true }
);

FoodEntrySchema.index({ stateId: 1, name: 1 }, { unique: true });

const FoodEntry = mongoose.model('FoodEntry', FoodEntrySchema);

export default FoodEntry;

```

API

The API in this project allows the frontend to communicate with the backend and interact with the database. It is mainly used to save and retrieve the diet planner state, including user data, meal plans, BMI results, and food items.

Files related to the API:

1. **backend/index.js** → Starts the server and registers the API routes.
2. **backend/routes/health.js** → Provides a health check endpoint to confirm the backend is running.
3. **backend/routes/state.js**→ Handles the main API requests for saving and loading state data.
4. **src/utils/api.js**→ Frontend file that sends requests to the backend API.

API description:

1. **GET /api/health** → Checks whether the server is running.
2. **GET /api/state** → Retrieves the saved planner state from the database.
3. **POST /api/state** → Saves or updates the planner state in the database.

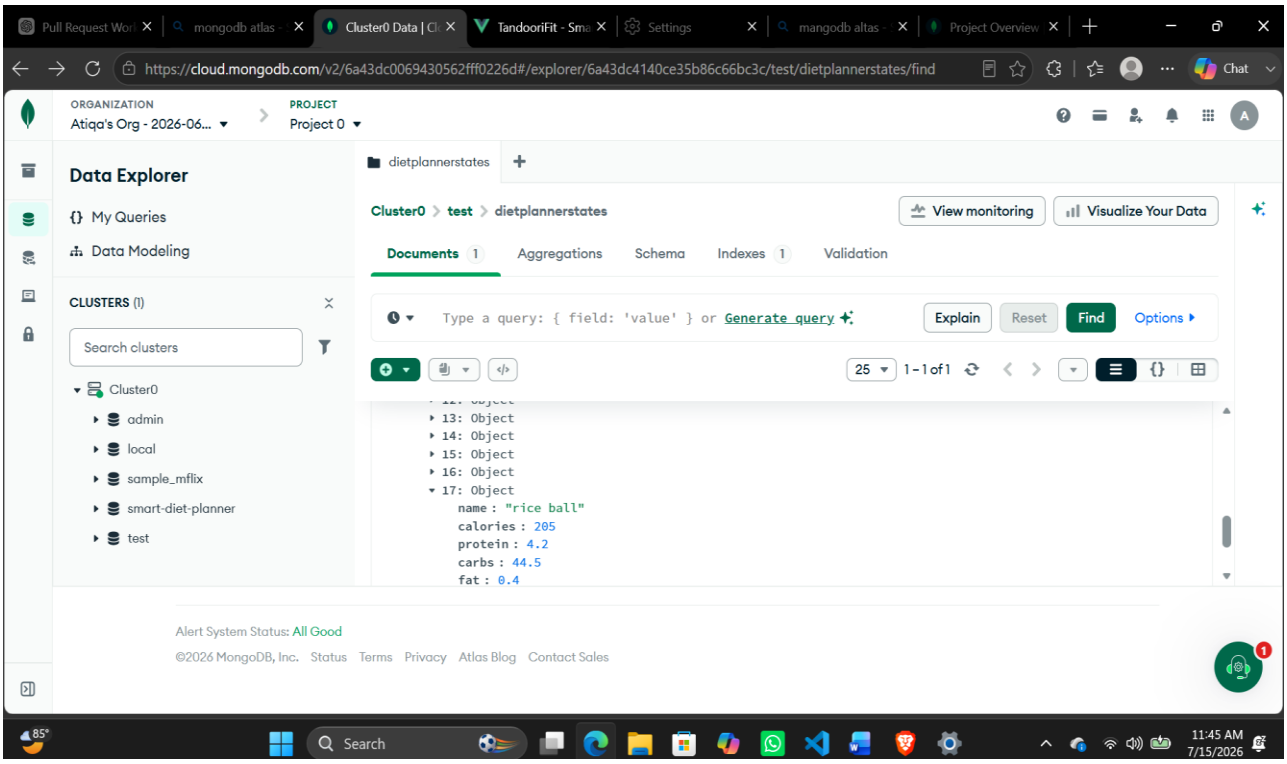
Example code of API (health.js)

```
import express from 'express';
import mongoose from 'mongoose';

const router = express.Router();

// GET /api/health
router.get('/', async (req, res) => {
  try {
    const mongoStatus = mongoose.connection.readyState === 1 ? 'connected' : 'disconnected';
    res.status(200).json({
      success: true,
      message: 'Server is running',
      mongodbStatus: mongoStatus,
      timestamp: new Date().toISOString(),
    });
  } catch (error) {
    res.status(500).json({ success: false, error: 'Health check failed' });
  }
});

export default router;
```

MongoDB

MongoDB is used in this project as the database system for storing the app's persistent data. It stores the diet planner state, including user information, meal plan details, BMI results, and food items, so that the data can be loaded again after a page refresh or reopening the app.

1. **.env** → contains the MongoDB connection string.
2. **backend/config/db.js** → connects the backend to the MongoDB database.
3. **backend/models/DietPlannerState.**js → defines the main document structure stored in MongoDB.
4. **backend/models/FoodItem.js** → defines the food item schema stored in MongoDB.
5. **backend/models/FoodEntry.js** → defines food entry records stored in MongoDB.
6. **backend/models/MealPlanItem.js** → defines meal plan entries stored in MongoDB.
7. **backend/models/Result.js** → defines BMI/BMR/TDEE result data stored in MongoDB.



Github Repository Link:

https://github.com/jasimjawediqbal/Smart-Diet-Planner

Vercel Deployment Link:

https://smart-diet-planner-iota.vercel.app/

Conclusion

The Smart Diet Planner frontend was developed using Vue 3 and Vite, focusing on a modern and modular single-page application architecture. The system uses Vue Router for smooth navigation between different views, while reusable components ensure a clean and organized code structure. Communication between components is handled using props and emits, following proper parent-child data flow principles. This frontend serves as a strong foundation for the project, and the backend integration will be developed in the next phase to extend its functionality. The Smart Diet Planner project is built with a frontend interface powered by Vue.js, a backend server using Node.js and Express, and a MongoDB database for persistent storage. The backend handles API requests, connects to MongoDB, and stores important data such as user details, meal plans, BMI results, and food items. The API acts as the communication bridge between the frontend and the database, allowing the app to save and retrieve information smoothly. Overall, this architecture makes the application functional, organized, and capable of maintaining data across sessions.